Pulled automatically principle 548
put() method 91

Q
QA (quality assurance) environment 24
quality attributes 524
queues 332, 349 – 351
QuoteController class 561
QuoteFunctions class 568
quote-function service 576
Quote record 559
QuoteServiceApplicationTests class 561
QuoteService class 560
quote-service Git repository 558

R
R2DBC (Reactive Relational Database Connectivity) 263
RabbitMQ 331 – 335
  AMQP for messaging systems 332
  configuring Spring integration with 344 – 346
  exploring exchanges and queues in 349 – 351
  running using Kubernetes Operator 596 – 597
  using for publish/subscribe communications 333 – 335
rabbitmq-system namespace 597
RANDOM_PORT Web environment 98
RateLimiterConfig class 315, 389
rate limiting 311 – 316
  configuring request rate limiter 314 – 316
  integrating Spring with Redis 313 – 314
  running Redis as container 312 – 313
RBAC (role-based access control) 373, 407 – 408, 421
RDS (Relational Database Service) 151
reactive applications 286 – 292
  testing data persistence with @DataR2dbcTest and Testcontainers 289 – 290
  testing REST clients with mock web server 287 – 289
  testing REST controllers with @WebFluxTest 290 – 292
ReactiveAuditorAware bean 438
ReactiveClientRegistrationRepository bean 393
reactive clients 273 – 279
  how to exchange data 275 – 276
  implementing REST clients with WebClient 276 – 279
  service-to-service communication in Spring 273 – 274
reactive functions 339 – 340
ReactiveJwtDecoder bean 432
Reactive Manifesto 255
Reactive Relational Database Connectivity (R2DBC) 263
reactive repositories 267
reactive servers 259 – 272
  bootstrapping reactive application with Spring Boot 261 – 263
  exposing REST API with Spring WebFlux 271 – 272
  implementing business logic with reactive streams 269 – 270